Command 인젝션 취약점 진단 및 대응

01 Command 인젝션 개념과 발생 원리

02 Command 인젝션 취약점 진단 및 대응

📌 학습 목표

- ✅ Command 인젝션 취약점의 개념과 발생 원리를 이해한다.
- ✅ 실제 공격 기법을 학습하고, 취약점을 진단하는 방법을 익힌다.
- ✅ 시큐어코딩 및 시스템 보호 기법으로 Command 인젝션을 방어할 수 있다.

📍 학습 내용

1. Command 인젝션 개념 및 발생원리
2. Command 인젝션 취약점 진단 및 대응

Command 인젝션 취약점 진단 및 대응

01 Command 인젝션 개념 및 발생 원리

02 Command 인젝션 취약점 진단 및 대응

Command 인젝션이란?

개요

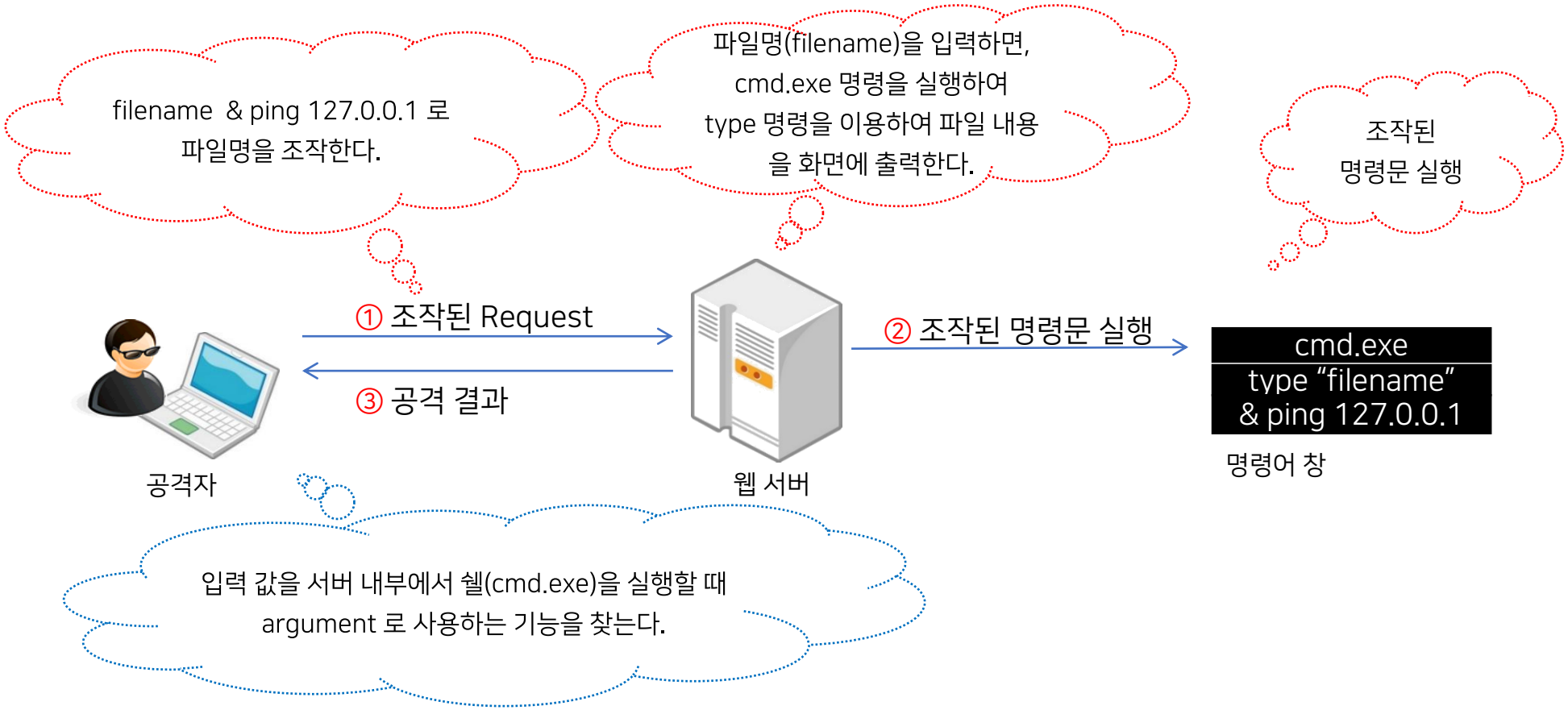
적절한 검증 절차를 거치지 않은 사용자 입력 값이 명령어의 일부 또는 전부로 구성되어 실행되는 경우 의도하지 않은 시스템 명령어가 실행되어 부적절하게 권한이 변경되거나 시스템 동작에 악영향을 미친다.

발생원인

운영체제 명령을 실행하는 함수를 사용하는 경우, 검증되지 않은 사용자 입력 값이 명령이나 명령의 파라미터로 사용되는 경우에 발생한다.



Command 인젝션이란?



운영체제별 명령어 실행 방식 이해 - Windows

기본 명령어 인터프리터

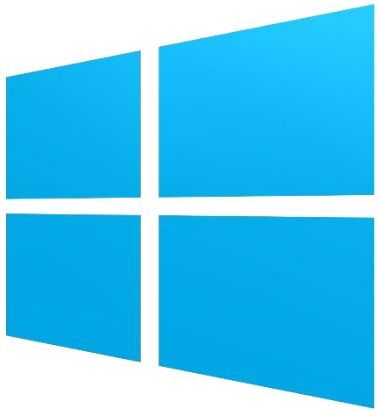
- 주로 cmd.exe와 최근에는 PowerShell 사용
- 내부 명령어와 외부 프로그램 호출 방식 구분

명령어 파싱 및 실행 과정

- 특수문자(예: &, |, >)를 이용한 명령어 연결 및 리다이렉션

보안 고려사항

- 명령어 인젝션 시, cmd.exe의 특수문자 해석을 악용 가능
- PowerShell의 스크립트 실행 정책 및 명령어 확장 취약점 존재



운영체제별 명령어 실행 방식 이해 - Linux

기본 명령어 인터프리터

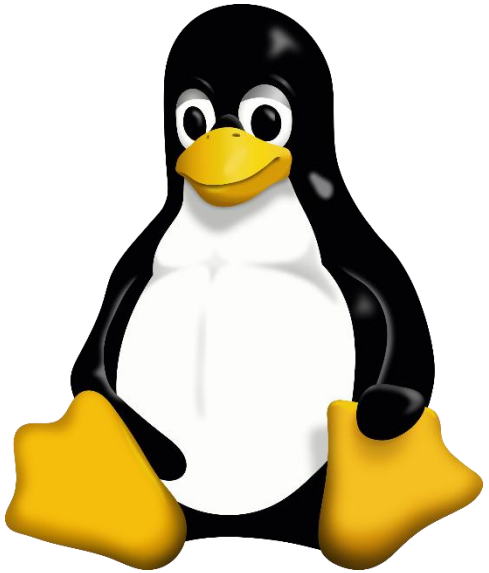
- 기본적으로 /bin/sh(대부분 Bash 기반) 사용
- 쉘의 확장 기능(파이프, 리다이렉션, 와일드카드 등) 활용

명령어 파싱 및 실행 과정

- 공백, 따옴표, 이스케이프 문자() 처리 방식
- 환경 변수, 명령어 치환(`, $() 등)의 해석 과정 포함

보안 고려사항

- 쉘 메타문자 및 파이프, 명령어 치환의 잘못된 처리로 인젝션 위험 증가
- 환경 변수 조작 및 경로 변조 공격 가능성



Command 인젝션 취약점 진단 및 대응

01 Command 인젝션 개념 및 발생 원리

02 Command 인젝션 취약점 진단 및 대응

Command 인젝션 취약점 진단 - 수동진단

수동 진단의 목표

사용자 입력이 시스템 명령어와 결합되어 실행되는 경우, 악의적인 조작이 가능한지 확인

의도치 않은 명령 실행이나 출력 조작을 통해 취약점을 탐지

기본 접근 방법

테스트 입력 값에 특수문자 및 제어 문자를 삽입하여 명령어의 실행 흐름 변화를 관찰

다양한 OS(Windows, Linux)와 쉘의 파싱 규칙을 고려하여 테스트 케이스 설계

Command 인젝션 취약점 진단 - 수동진단

명령어 연결자 테스트

&, &&, ||, ; 등으로 추가 명령어 실행 테스트

예) "test && cat /etc/passwd" 또는 "test; ls"

파이프 및 리다이렉션 테스트

|, >, < 등을 활용해 명령어 조작 테스트

예) "test | ls", "test > output.txt"

Command 인젝션 취약점 진단 - 수동진단

환경 변수 활용 테스트

환경 변수 활용

$PATH, $IFS 등을 이용하여 명령어 실행 환경 변경하거나, 검증 우회 시도

예) `"test $PATH"` 또는 "test$IFSls" 처럼 환경변수 활용 가능한지 테스트

서브 명령 실행 테스트

백틱 및 $() 사용

백틱 (`) 또는 $()`를 사용해 서브 명령어 실행되는지 테스트

예) `"test `ls`"` 또는 `"test $(ls)"`

Command 인젝션 취약점 진단 – 자동화도구 활용



ZAP (Zed Attack Proxy) 설치 및 기본 설정

- ZAP 공식 사이트(https://www.zaproxy.org/)에서 최신 버전 다운로드 및 설치
- ZAP 실행 후, 프록시 포트(기본 8080) 확인

진단 환경 구성

- 진단 대상 웹 서비스 준비
- 브라우저 프록시 설정
  - 브라우저의 프록시 설정을 ZAP의 프록시(예: localhost:8080)로 변경하여 모든 트래픽이 ZAP을 통해 흐르도록 설정



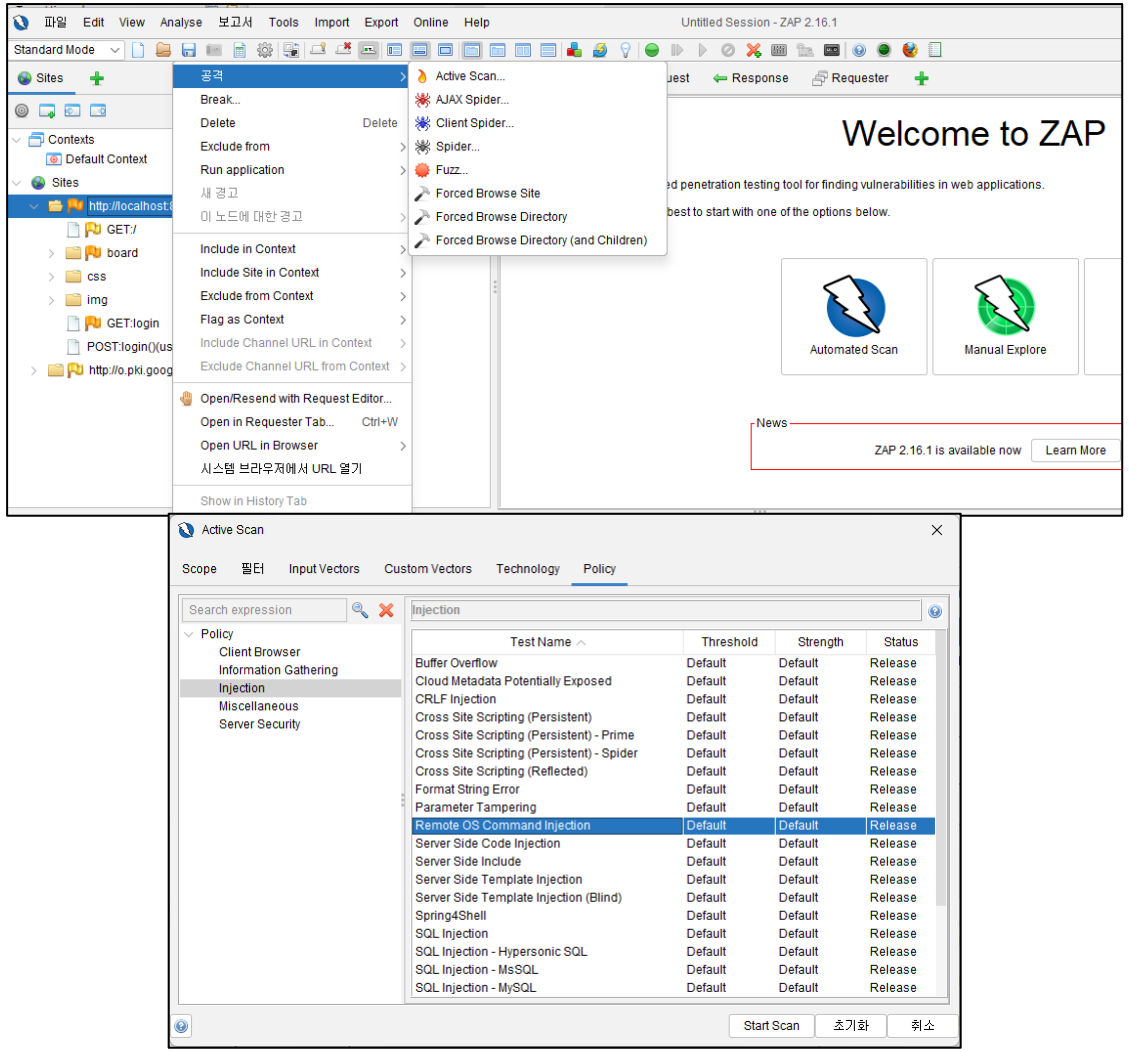
Command 인젝션 취약점 진단 – 자동화도구 활용

ZAP의 자동 검사 Active Scan 수행 방법

ZAP 좌측 트리에서 테스트 대상 URL 선택 후, 우 클릭
→ "Attack" → "Active Scan" 실행

스캔 결과 확인

스캔 결과 창에서 "Command Injection" 관련 알림
(예: 에러 메시지, 비정상 응답 코드 등) 확인
알림 상세 정보를 통해 의심되는 입력 필드 및 실행 결과
분석



Command 인젝션 대응 – 시큐어코딩 적용

명령어 삽입 취약점이 발생하는 경우

안전하지 않은 코드

```
...
 String version = request.getParameter("version");
 String cmd = new String("cmd.exe /K \"run.bat \"");
 Runtime.getRuntime().exec(cmd + " c:\\prog_cmd\\" + version);
...
```

안전한 코드

```
...
 String version = request.getParameter("version");
 if(version.matches("[0-9.]+")) {
   Runtime.getRuntime().exec(new String[]{"cmd.exe","/c", "run.bat", version});
 } else {
    부적합한 값으로 처리
 }
…
```

Command 인젝션 대응 – 시큐어코딩 적용

안전한 API 사용

직접 OS 명령어를 실행하지 않고, 안전하게 프로세스를 실행하는 API (예: 인자 배열 방식의 ProcessBuilder, execvp() 등) 사용

→ Command 인젝션 위험을 줄이기 위해 파라미터와 명령어를 명확하게 분리

사용자 입력 값 검증

화이트리스트 기반 검증: 허용된 값만 받아들이고, 예상치 못한 입력은 모두 거부

→ 정규 표현식, 타입 체크 등을 활용하여 올바른 포맷만 통과

메타 문자 필터링

사용자 입력에서 특수문자 (&, ;, |, >, <, $(), ``` 등)를 제거하거나 이스케이프 처리

→ 입력 값에 포함된 메타문자를 명시적으로 치환하거나 제거하여, 명령어 결합 우회 방지

Command 인젝션 대응 – 시큐어코딩 적용

에러 핸들링 및 로깅

입력 값 검증 실패 및 명령어 실행 오류 시, 민감 정보가 노출되지 않도록 안전하게 처리

→ 오류 발생 시 내부 로그에만 기록하고, 사용자에게는 최소한의 정보만 제공

코드 리뷰 및 정적 분석

정기적인 코드 리뷰와 정적 분석 도구를 활용하여 Command 인젝션 취약점 사전 탐지

보안 관련 라이브러리 및 프레임워크의 최신 업데이트 적용

테스트 케이스 마련

다양한 악의적 입력(우회 페이로드, 인코딩 변형 등)을 포함한 단위 테스트, 통합 테스트를 수행

Command 인젝션 대응 – 시스템 보호 전략

1. 최소 권한 원칙 적용

   애플리케이션 또는 명령어 실행 시 비 권한 사용자 계정을 활용

   시스템 서비스나 웹 서버의 실행 권한 제한을 통해 공격 성공 시 피해 범위 최소화

2. 애플리케이션 샌드박싱 및 컨테이너화

   샌드박스 환경 또는 컨테이너(예: Docker)를 이용해 애플리케이션 격리

   외부 명령 실행 시 격리된 환경 내에서 실행하여 시스템 전체에 영향을 주지 않도록 구성

3. 로깅 및 모니터링 활성화

   명령어 실행 로그, 에러 로그, 접근 로그를 체계적으로 기록

   이상 행동 감지 시 즉각 알림 및 대응할 수 있는 모니터링 시스템 구축

요약 정리

- Command 인젝션은 검증되지 않은 입력이 OS 명령어를 조작해 시스템을 장악하는 취약점입니다.
- 공격 원리는 메타문자(;, &, |)를 사용해 OS 명령어 파싱 방식을 악용하는 방식입니다.
- 취약점 진단은 수동 진단과 OWASP ZAP 같은 자동화 도구를 병행해 수행합니다.
- 대응 방안으로 입력 값 검증과 안전한 API 사용, 그리고 최소 권한·샌드박싱·모니터링을 사용합니다.
- 지속적으로 코드 리뷰와 CI/CD 보안 검증을 적용해 사전에 차단하고 재발을 방지해야 합니다.

참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어

감사합니다.



퀴즈

1. 다음 중 Windows 환경에서 Command 인젝션 위험을 줄이기 위한 방법으로 가장 옳은 것은 ?

1) 문자열 결합으로 명령어를 만들고, 관리자 권한으로 실행한다.
2) 사용자 입력을 전혀 검증하지 않고 바로 cmd.exe /c에 전달한다.
3) 명령 인자를 배열로 분리하고, 권한이 제한된 계정으로 실행한다.
4) PowerShell 스크립트를 항상 시스템 권한으로만 실행한다.

2. 다음 중 Command 인젝션 공격 예시로 볼 수 있는 것은 ?

1) `ls -al`
2) `dir C:\Users`
3) `ls; rm -rf /`
4) `ping 127.0.0.1`

3. Linux 환경에서 명령어를 연속 실행하기 위해 자주 사용되는 쉘 연산자 기호를 정확히 쓰세요.

( )



퀴즈 정답

1. **다음 중 Windows 환경에서 Command 인젝션 위험을 줄이기 위한 방법으로 가장 옳은 것은 ?**

   3) 명령 인자를 배열로 분리하고, 권한이 제한된 계정으로 실행한다.

   인자를 배열로 분리하고, 최소 권한 계정을 사용해야 인젝션 위험이 낮아집니다.

2. **다음 중 Command 인젝션 공격 예시로 볼 수 있는 것은 ?**

   3) `ls; rm -rf /`

   세미콜론 ;을 사용해 기존 명령 뒤에 rm -rf /를 추가로 실행하므로, 전형적인 인젝션 공격 시도입니다.

3. **Linux 환경에서 명령어를 연속 실행하기 위해 자주 사용되는 쉘 연산자 기호를 정확히 쓰세요.**

   (;)